

Continuous docker building

1

Github Actions and Docker Automation

- Github Actions can do more than run tests; it can build Docker images continuously.
- Docker builds can take time, so automating image creation on code commits is useful.
- Automation can speed up processes using parallelized online compute power.
- Docker Hub is a cloud solution for storing and pulling Docker images, free for public use.
- This session focuses on automating Docker builds and pushing them to Docker Hub.
- Future module will explore the same process for general cloud providers.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

2

? Exercises

For these exercises you can choose to work with any docker file of your choosing. If you want an easy docker file, you can use the following:

```
FROM busybox  
CMD echo "Howdy cowboy"
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

3

Guidelines

1. Pushing Docker File to Repository
2. Creating a Docker Hub Account
3. Generating Docker Hub Access Token
4. Configuring GitHub Repository with Secrets
5. Creating GitHub Actions Workflow
6. Uploading Workflow and Executing Build
7. Pulling Docker Image Locally
8. Optional: Running Container in GitHub



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

4

1. Start by pushing whatever docker file you want that should be continuously build to your repository
2. Start by creating a [Docker Hub account](#)
3. Next, within Docker Hub create an access token by going to Settings -> Security. Click the New Access Token button and give it a name that you recognize.
4. Copy the newly created access token and head over to your Github repository online. Go to Settings -> Secrets -> Actions and click the New repository secret. Copy over the access token and give it the name DOCKER_HUB_TOKEN. Additionally, add two other secrets DOCKER_HUB_USERNAME and DOCKER_HUB_REPOSITORY that contains your docker username and docker repository name respectively.



5. Next we are going to construct the actual Github actions workflow file:

```
name: Docker Image continuous integration

on:
  push:
    branches: [ master ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Build the Docker image
        run: |
          echo "${{ secrets.DOCKER_HUB_TOKEN }}" | docker login \
            -u "${{ secrets.DOCKER_HUB_USERNAME }}" --password-stdin
          docker build . --file Dockerfile \
            --tag docker.io/${{ secrets.DOCKER_HUB_USERNAME }}/${{
secrets.DOCKER_HUB_REPOSITORY }}:${GITHUB_SHA}
          docker push docker.io/${{ secrets.DOCKER_HUB_USERNAME }}/${{
secrets.DOCKER_HUB_REPOSITORY }}:${GITHUB_SHA}
```



Analyzing the Workflow File

- The first part of the workflow file should be familiar.
- Focus on the last three lines for key functionality.
- Review each line and understand its purpose.
- Refer to the help pages for:
 - `docker login`
 - `docker build`
 - `docker push`



7

6. Upload the workflow to your GitHub repository and check that it is being executed. If everything you should be able to see the the build docker image in your container repository in docker hub.
7. Make sure that you can execute `docker pull` locally to pull down the image that you just continuously build
8. (Optional) To test that the container works directly in GitHub you can also try to include an additional step that actually runs the container.

```
- name: Run container
  run: |
    docker run ...
```



8